



[Bastion Cleve Magdeburg]

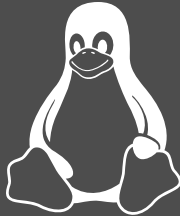
How Configurable is the Linux Kernel?

Analyzing Two Decades of Feature-Model History

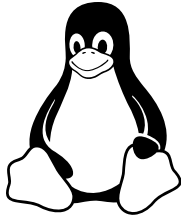
SE 2026 (TOSEM 2025) — February 23–27 — Bern, Switzerland

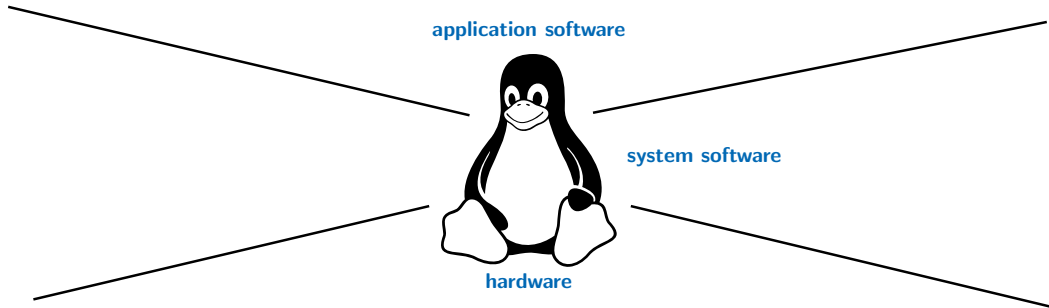
Elias Kuitert¹, Chico Sundermann³, Thomas Thüm³, Tobias Heß², Sebastian Krieter³, Gunter Saake¹

University of Magdeburg¹, Ulm², Braunschweig³, Germany



Motivation and Background



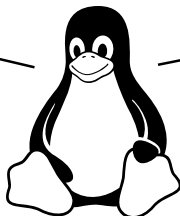


	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W906	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabían Murillo	6/25/1963
9	R186	Eliás Pihuave	2/16/1967
10	F470	Iván Mendigeta	7/9/1963

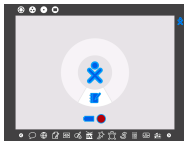
application software

system software

hardware



	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W906	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabian Murillo	6/25/1963
9	R186	Elias Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



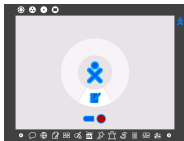
application software

system software

hardware



	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W906	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabían Murillo	6/25/1963
9	R186	Eliás Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



application software

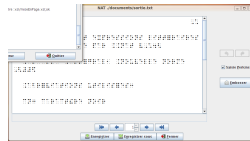
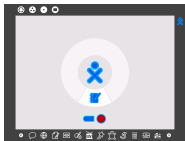


system software

hardware



	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W906	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabían Murillo	6/25/1963
9	R186	Eliás Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



application software

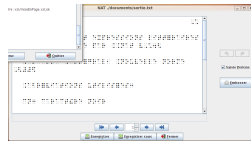
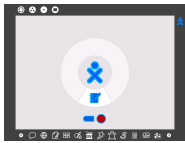


system software

hardware



	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabían Murillo	6/25/1963
9	R186	Eliás Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



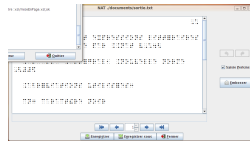
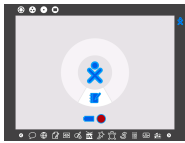
application software

system software

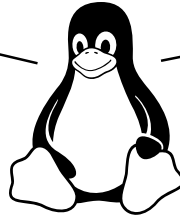
hardware



	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabían Murillo	6/25/1963
9	R186	Eliás Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



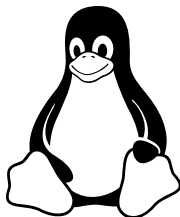
application software



system software

hardware





KConfig Feature Model

...

`config TOSHIBA`
`tristate "Toshiba Laptop"`
`depends on X86_32`
`help`

Toshiba portables with a genuine Toshiba BIOS

`config OLPC`
`bool "One Laptop Per Child"`
`depends on !X86_PAE`
`help`

Unique features of the OLPC XO hardware

`config SUPERH`
`def_bool y`
`select ARCH_32BIT_OFF`
`help`

Sega Dreamcast gaming console

`config X86_32_IRIS`
`tristate "Eurobraille/Iris"`
`depends on X86_32`
`help`

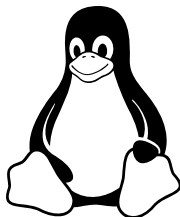
Iris machines from EuroBraille

`config X86_GOLDFISH`
`bool "Goldfish"`
`depends on X86_EXT`
`help`

Goldfish virtual platform for Android development

Feature-to-Code Mapping

...



KConfig Feature Model

...

`config TOSHIBA`
`tristate "Toshiba Laptop"`
`depends on X86_32`
`help`
Toshiba portables with a genuine Toshiba BIOS

`config OLPC`
`bool "One Laptop Per Child"`
`depends on !X86_PAE`
`help`
Unique features of the OLPC XO hardware

`config SUPERH`
`def_bool y`
`select ARCH_32BIT_OFF`
`help`
Sega Dreamcast gaming console

`config X86_32_IRIS`
`tristate "Eurobraille/Iris"`
`depends on X86_32`
`help`
Iris machines from EuroBraille

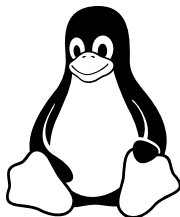
`config X86_GOLDFISH`
`bool "Goldfish"`
`depends on X86_EXT`
`help`
Goldfish virtual platform for Android development

Feature-to-Code Mapping

...

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

KBuild Makefiles



KConfig Feature Model

...

`config TOSHIBA`
`tristate "Toshiba Laptop"`
`depends on X86_32`
`help`
Toshiba portables with a genuine Toshiba BIOS

`config OLPC`
`bool "One Laptop Per Child"`
`depends on !X86_PAE`
`help`
Unique features of the OLPC XO hardware

`config SUPERH`
`def_bool y`
`select ARCH_32BIT_OFF`
`help`
Sega Dreamcast gaming console

`config X86_32_IRIS`
`tristate "Eurobraille/Iris"`
`depends on X86_32`
`help`
Iris machines from EuroBraille

`config X86_GOLDFISH`
`bool "Goldfish"`
`depends on X86_EXT`
`help`
Goldfish virtual platform for Android development

Feature-to-Code Mapping

...

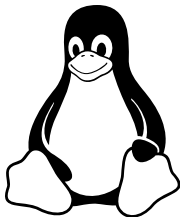
```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA  
#include <linux/toshiba.h>  
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA  
SMMRegisters regs;  
regs.eax = 0xff00; /* HCI_SET */  
regs.ebx = 0x0002; /* HCI_BACKLIGHT */  
regs.ecx = 0x0000; /* HCI_DISABLE */  
tosh_smm(&regs);  
#endif
```



KConfig Feature Model

...

config TOSHIBA
tristate "Toshiba Laptop"
depends on X86_32
help
Toshiba portables with a
genuine Toshiba BIOS

config OLPC
bool "One Laptop Per Child"
depends on !X86_PAE
help
Unique features of the OLPC
XO hardware

config SUPERH
def_bool y
select ARCH_32BIT_OFF
help
Sega Dreamcast gaming
console

config X86_32_IRIS
tristate "Eurobraille/Iris"
depends on X86_32
help
Iris machines from
EuroBraille

config X86_GOLDFISH
bool "Goldfish"
depends on X86_EXT
help
Goldfish virtual platform
for Android development

Feature-to-Code Mapping

```
obj-${CONFIG_TOSHIBA} += toshiba.o
```

KBuild Makefiles

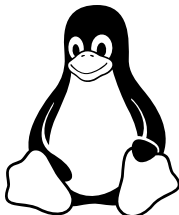
```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
SMMRegisters regs;
regs.eax = 0xff00; /* HCI_SET */
regs.ebx = 0x0002; /* HCI_BACKLIGHT */
regs.ecx = 0x0000; /* HCI_DISABLE */
tosh_smm(&regs);
#endif
```

make menuconfig

```
.config - Linux/x86 6.19.0-rc7 Kernel Configuration
+ Processor type and features
  Processor type and features
  Arrow keys navigate the menu. <Enter> selects submenus
  ↑(-)
  [*] IOAPIC and IOPL Emulation
  <+> Toshiba Laptop support
  [ ] Enable X86 board specific fixups for reboot (NEW)
  ↓(+)
  <Select> < Exit > < Help > < Save > < Load >
```



KConfig Feature Model

```
config TOSHIBA
tristate "Toshiba Laptop"
depends on X86_32
help
  Toshiba portables with a
  genuine Toshiba BIOS
```

```
config OLPC
bool "One Laptop Per Child"
depends on !X86_PAE
help
  Unique features of the OLPC
  XO hardware
```

```
config SUPERH
def_bool y
select ARCH_32BIT_OFF
help
  Sega Dreamcast gaming
  console
```

```
config X86_32_IRIS
tristate "Eurobraille/Iris"
depends on X86_32
help
  Iris machines from
  EuroBraille
```

```
config X86_GOLDFISH
bool "Goldfish"
depends on X86_EXT
help
  Goldfish virtual platform
  for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

KBuild Makefiles

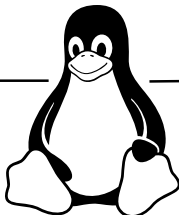
```
#ifdef CONFIG_TOSHIBA  
#include <linux/toshiba.h>  
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA  
SMMRegisters regs;  
regs.eax = 0xff00; /* HCI_SET */  
regs.ebx = 0x0002; /* HCI_BACKLIGHT */  
regs.ecx = 0x0000; /* HCI_DISABLE */  
tosh_smm(&regs);  
#endif
```

make menuconfig

```
.config - Linux/x86 6.19.0-rc7 Kernel Configuration  
+ Processor type and features  
  Processor type and features  
  Arrow keys navigate the menu. <Enter> selects submenus  
  (-) ~~~~~  
  [*] IOAPIC and IOPL Emulation  
  <+> Toshiba Laptop support  
  [ ] Enable X86 board specific fixups for reboot (NEW)  
  (+) ~~~~~  
  <Select> < Exit > < Help > < Save > < Load >
```



KConfig Feature Model

```
config TOSHIBA  
tristate "Toshiba Laptop"  
depends on X86_32  
help  
Toshiba portables with a  
genuine Toshiba BIOS
```

```
config OLPC  
bool "One Laptop Per Child"  
depends on !X86_PAE  
help  
Unique features of the OLPC  
XO hardware
```

```
config SUPERH  
def_bool y  
select ARCH_32BIT_OFF  
help  
Sega Dreamcast gaming  
console
```

```
config X86_32_IRIS  
tristate "Eurobraille/Iris"  
depends on X86_32  
help  
Iris machines from  
EuroBraille
```

```
config X86_GOLDFISH  
bool "Goldfish"  
depends on X86_EXT  
help  
Goldfish virtual platform  
for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

KBuild Makefiles

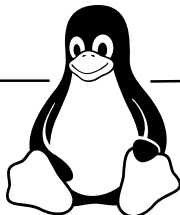
```
#ifdef CONFIG_TOSHIBA  
#include <linux/toshiba.h>  
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA  
SMMRegisters regs;  
regs.eax = 0xff00; /* HCI_SET */  
regs.ebx = 0x0002; /* HCI_BACKLIGHT */  
regs.ecx = 0x0000; /* HCI_DISABLE */  
tosh_smm(&regs);  
#endif
```

make menuconfig

```
.config - Linux/x86 6.19.0-rc7 Kernel Configuration  
+ Processor type and features  
  Processor type and features  
  Arrow keys navigate the menu. <Enter> selects submenus  
  (-) ~~~~~  
  [*] IOAPIC and IOPL Emulation  
  <+> Toshiba Laptop support  
  [ ] Enable X86 board specific fixups for reboot (NEW)  
  (+) ~~~~~  
  <Select> < Exit > < Help > < Save > < Load >
```



Linux is a
software product line

KConfig Feature Model

```
config TOSHIBA  
tristate "Toshiba Laptop"  
depends on X86_32  
help  
Toshiba portables with a  
genuine Toshiba BIOS
```

```
config OLPC  
bool "One Laptop Per Child"  
depends on !X86_PAE  
help  
Unique features of the OLPC  
XO hardware
```

```
config SUPERH  
def_bool y  
select ARCH_32BIT_OFF  
help  
Sega Dreamcast gaming  
console
```

```
config X86_32_IRIS  
tristate "Eurobraille/Iris"  
depends on X86_32  
help  
Iris machines from  
EuroBraille
```

```
config X86_GOLDFISH  
bool "Goldfish"  
depends on X86_EXT  
help  
Goldfish virtual platform  
for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

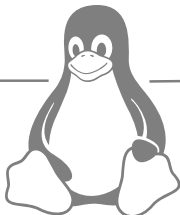
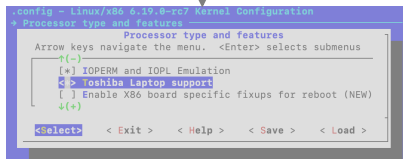
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
SMMRegisters regs;
regs.eax = 0xff00; /* HCI_SE
regs.ebx = 0x0002; /* HCI_BA
regs.ecx = 0x0000; /* HCI_DI
tosm_smm(&regs);
#endif
```

make menuconfig



Linux is a
software product

KConfig Feature Model

```
config TOSHIBA
tristate "Toshiba Laptop"
depends on X86_32
help
Toshiba portables with a
genuine Toshiba BIOS
```

```
config OLPC
bool "One Laptop Per Child"
depends on !X86_PAE
help
Unique features of the OLPC
XO hardware
```

```
config SUPERH
def_bool y
select ARCH_32BIT_OFF
help
Sega Dreamcast gaming
console
```

```
config X86_32_IRIS
tristate "Eurobraille/Iris"
depends on X86_32
help
Iris machines from
EuroBraille
```

```
config X86_GOLD
bool "Goldfish"
depends on X86
help
Goldfish virt
for Android d
```



Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

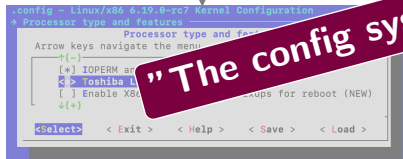
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA  
#include <linux/toshiba.h>  
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA  
SMMRegisters regs;  
regs.eax = 0xff00; /* HCI_SET */  
regs.ebx = 0x0002; /* HCI_BACKLIGHT */  
regs.ecx = 0x0000; /* HCI_DISABLE */  
h_smm(&regs);
```

make menuconfig



"The config system is a nightmare"



Linux is a
software product line

KConfig Feature Model

```
config TOSHIBA  
tristate "Toshiba Laptop"  
depends on X86_32  
help  
Toshiba portables with a  
genuine Toshiba BIOS
```

```
config OLPC  
bool "One Laptop Per Child"  
depends on !X86_PAE  
help  
Unique features of the OLPC  
XO hardware
```

```
config SUPERH  
def_bool y  
select ARCH_32BIT_OFF  
help  
Sega Dreamcast gaming  
console
```

```
config X86_32_IRIS  
tristate "Eurobraille/Iris"  
depends on X86_32  
help  
Iris machines from  
EuroBraille
```

```
config X86_GOLDFISH  
bool "Goldfish"  
depends on X86_EXT  
help  
Goldfish virtual platform  
for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

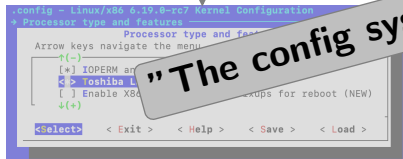
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA  
#include <linux/toshiba.h>  
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA  
SMMRegisters regs;  
regs.eax = 0xff00; /* HCI_SET */  
regs.ebx = 0x0002; /* HCI_BACKLIGHT */  
regs.ecx = 0x0000; /* HCI_DISABLE */  
_smm(&regs);  
#endif
```

make menuconfig



"The config system is a nightmare"

"It's gotten out of control"



Linux is a
software product line

KConfig Feature Model

```
config TOSHIBA  
tristate "Toshiba Laptop"  
depends on X86_32  
help  
Toshiba portables with a  
genuine Toshiba BIOS
```

```
config OLPC  
bool "One Laptop Per Child"  
depends on !X86_PAE  
help  
Unique features of the OLPC  
XO hardware
```

```
config SUPERH  
def_bool y  
select ARCH_32BIT_OFF  
help  
Sega Dreamcast gaming  
console
```

```
config X86_32_IRIS  
tristate "Eurobraille/Iris"  
depends on X86_32  
help  
Iris machines from  
EuroBraille
```

```
config X86_GOLDFISH  
bool "Goldfish"  
depends on X86_EXT  
help  
Goldfish virtual platform  
for Android development
```


Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA  
#include <linux/toshiba.h>  
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA  
SMMRegisters regs;  
regs.eax = 0xff00; /* HCI_SET */  
regs.ebx = 0x0002; /* HCI_BACKLIGHT */  
regs.ecx = 0x0000; /* HCI_DISABLE */  
_smm(&regs);
```

make menuconfig



.config - Linux/x86 6.19.0-rc7 Kernel Configuration
+ Processor type and features
Processor type and features
Arrow keys navigate the menu
↑(-) IOPERM and
[<] Toshiba Laptop
[>] Enable X86
↓(+)
<Select> <Exit> <Help>

"The config system is a nightmare"

"There are simply already far too many features"

software product line

KConfig Feature Model

```
config TOSHIBA  
tristate "Toshiba Laptop"  
depends on X86_32  
help  
Toshiba portables with a  
genuine Toshiba BIOS
```

```
config OLPC  
bool "One Laptop Per Child"  
depends on !X86_PAE  
help  
Unique features of the OLPC  
XO hardware
```

```
config SUPERH  
def_bool y  
select ARCH_32BIT_OFF  
help  
Sega Dreamcast gaming  
console
```

```
config X86_32_IRIS  
tristate "Eurobraille/Iris"  
depends on X86_32  
help  
Iris machines from  
EuroBraille
```

```
config X86_GOLDFISH  
bool "Goldfish"  
depends on X86_EXT  
help  
Goldfish virtual platform  
for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

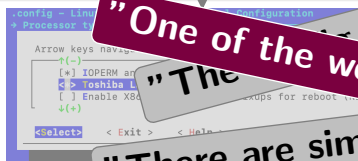
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA  
#include <linux/toshiba.h>  
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA  
SMMRegisters regs;  
regs.eax = 0xff00; /* HCI_SET */  
regs.ebx = 0x0002; /* HCI_BACKLIGHT */  
regs.ecx = 0x0000; /* HCI_DISABLE */  
_smm(&regs);
```

make menuconfig



"One of the worst parts of the whole project"

"The system is a nightmare"

"It's gotten out of control"

"There are simply already too many features"

"There are simply already too many features"

KConfig Feature Model

```
config TOSHIBA  
tristate "Toshiba Laptop"  
depends on X86_32  
help  
Toshiba portables with a  
genuine Toshiba BIOS
```

```
config OLPC  
bool "One Laptop Per Child"  
depends on !X86_PAE  
help  
Unique features of the OLPC  
XO hardware
```

```
config SUPERH  
def_bool y  
select ARCH_32BIT_OFF  
help  
Sega Dreamcast gaming  
console
```

```
config X86_32_IRIS  
tristate "Eurobraille/Iris"  
depends on X86_32  
help  
Iris machines from  
EuroBraille
```

```
config X86_GOLDFISH  
bool "Goldfish"  
depends on X86_EXT  
help  
Goldfish virtual platform  
for Android development
```

[Linus Torvalds]

Literature on the Linux Kernel's Configurability

Year	Revision	Architecture	Reported #Features	Reported #Configurations
2002	v2.5.45	*	—	—
2005	v2.6.12	i386	3,284	—
2008	v2.6.28	*	5,426	—
	v2.6.28	x86	5,321 5,323 6,888	—
	v2.6.28	x86?	5,701 6,888	—
	v2.6.28?	x86?	6,888	—
2009	v2.6.32	x86	6,319 6,320 60,072	—
2010	v2.6.33	x86	6,467 6,559 6,918 62,482	—
	v2.6.33?	x86?	5,913	—
2011	v3.1	*	11,691	—
2015	v4.0	x86	11,135	—
2016	v4.4	x86	15,500	—
2018	v4.18	x86	13,379 22,352	—
2020	v5.8	x86	14,817	—
2024	v6.7	*	—	—

(references omitted)

Literature on the Linux Kernel's Configurability

Year	Revision	Architecture	Reported #Features      	Reported #Configurations
2002	v2.5.45	*	—	—
2005	v2.6.12	i386	3,284	—
2008	v2.6.28	*	5,426	—
	v2.6.28	x86	5,321 5,323 6,888	—
	v2.6.28	x86?	5,701 6,888	—
	v2.6.28?	x86?	6,888	—
2009	v2.6.32	x86	6,319 6,320 60,072	—
2010	v2.6.33	x86	6,467 6,559 6,918 62,482	—
	v2.6.33?	x86?	5,913	—
2011	v3.1	*	11,691	—
2015	v4.0	x86	11,135	—
2016	v4.4	x86	15,500	—
2018	v4.18	x86	13,379 22,352	—
2020	v5.8	x86	14,817	—
2024	v6.7	*	—	—

#Features



varies wildly $\Rightarrow$ impacts analyses
unknown for recent revisions

(references omitted)

Literature on the Linux Kernel's Configurability

Year	Revision	Architecture	Reported #Features 	Reported #Configurations 
2002	v2.5.45	*	—	—
2005	v2.6.12	i386	3,284	—
2008	v2.6.28	*	5,426	—
	v2.6.28	x86	5,321 5,323 6,888	—
	v2.6.28	x86?	5,701 6,888	—
	v2.6.28?	x86?	6,888	—
2009	v2.6.32	x86	6,319 6,320 60,072	—
2010	v2.6.33	x86	6,467 6,559 6,918 62,482	—
	v2.6.33?	x86?	5,913	—
2011	v3.1	*	11,691	—
2015	v4.0	x86	11,135	—
2016	v4.4	x86	15,500	—
2018	v4.18	x86	13,379 22,352	—
2020	v5.8	x86	14,817	—
2024	v6.7	*	—	—

(references omitted)

#Features



varies wildly $\Rightarrow$ impacts analyses
unknown for recent revisions

#Configurations



completely **unknown**

Literature on the Linux Kernel's Configurability

Year	Revision	Architecture	Reported	Reported configurations
		Why Configurability?		
2002	v2.5			
2005	v2.6			
2008	v2.6			
	v2.6			
	v2.6			
	v2.6			
2009	v2.6			
2010	v2.6			
	v2.6			
2011	v3.2			
2015	v4.0			
2016	v4.4			
2018	v4.19			
2020	v5.8			
2024	v6.7	*	—	—

(references omitted)

#Features



varies wildly ⇒ impacts analyses
unknown for recent revisions

#Configurations



completely unknown

Literature on the Linux Kernel's Configurability

Year	Revision	Architecture	Reported	Reported configurations
Why Configurability? • judge complexity of feature models [Kuiter et al. 2024] • #features is fundamental, often stated in papers • #cfg less known, but equally useful [Sundermann et al. 2021]				
2002	v2.4			
2005	v2.6			
2008	v2.6			
	v2.6			
	v2.6			
	v2.6			
2009	v2.6			
2010	v2.6			
	v2.6			
2011	v3.2			
2015	v4.0			
2016	v4.4			
2018	v4.19			
2020	v5.8			
2024	v6.7	*	—	—

(references omitted)

#Features



varies wildly ⇒ impacts analyses
unknown for recent revisions

#Configurations



completely unknown

Literature on the Linux Kernel's Configurability

Year	Revision	Architecture	Reported	Reported configurations
Why Configurability?				
2002	v2.4		• judge complexity of feature models [Kuiter et al. 2024]	
2005	v2.6		• #features is fundamental, often stated in papers	
2008	v2.6		• #cfg less known, but equally useful [Sundermann et al. 2021]	
	v2.6		• needed for automated analyses based on SAT/#SAT	
2009	v2.6		interactive product configuration anomaly detection and explanation feature-model evolution and slicing modularization implicit constraints code parsing dead code analysis code simplification type checking model checking formal verification static code analysis dataflow analysis testing t-wise sampling uniform random sampling optimization of non-functional properties refactoring economical estimations ...	
2010	v2.6			
	v2.6			
2011	v3.2			
2015	v4.0			
2016	v4.4			
2018	v4.19			
2020	v5.8			
2024	v6.7	*	—	—

(references omitted)

#Features



varies wildly ⇒ impacts analyses
unknown for recent revisions

#Configurations



completely **unknown**

Literature on the Linux Kernel's Configurability

Year	Revision	Architecture	Reported #Features	Reported #Configurations
2002	v2.5.45	*	—	—
2005	v2.6.12	i386	3,284	—
2008	v2.6.28	*	5,426	—
	v2.6.28	x86	5,321 5,323 6,888	—
	v2.6.28	x86?	5,701 6,888	—
	v2.6.28?	x86?	6,888	—
2009	v2.6.32	x86	6,319 6,320 60,072	—
2010	v2.6.33	x86	6,467 6,559 6,918 62,482	—
	v2.6.33?	x86?		
2011	v3.1	*		
2015	v4.0	x86		
2016	v4.4	x86		
2018	v4.18	x86		
2020	v5.8	x86		
2024	v6.7	*	—	—

Our Goals

- which **factors** influence this?
- which results are **accurate**?
- when is it **too hard**?

#Features



varies wildly ⇒ impacts analyses
unknown for recent revisions

#Configurations



completely **unknown**

(references omitted)



Choosing and Analyzing the Feature Model

“are we talking about **the same** feature model?”

Choosing and Analyzing the Feature Model

“are we talking about **the same** feature model?”

1. Source Tree

⇒ here: **mainline kernel** (/torvalds/Linux)

Name

pub/scm/bluetooth

bluez.git

sbc.git

pub/scm/devel

pahole/pahole.git

sparse/sparse-dev.git

sparse/sparse.git

pub/scm/docs

docsko/backports.git

docsko/ieee1394.git

docsko/korg.git

docsko/mtk.git

docsko/nvdimmem.git

kernel/ksmap.git

kernel/pgpkeys.git

kernel/website.git

man-pages/man-pages.git

man-pages/website.git

tab/tab.git

Choosing and Analyzing the Feature Model

“are we talking about **the same** feature model?”

1. Source Tree

⇒ here: **mainline kernel** (torvalds/Linux)



2. Revision

⇒ here: **all releases** in 2002–2024 (i.e., KConfig)

Tags

v6.19 ...

🕒 2 weeks ago 🔗 05f7e89 📦 zip 📦 tar.gz

v6.19-rc8 ...

🕒 3 weeks ago 🔗 18f7fcd 📦 zip 📦 tar.gz

v6.19-rc7 ...

🕒 last month 🔗 63804fe 📦 zip 📦 tar.gz

Choosing and Analyzing the Feature Model

“are we talking about **the same** feature model?”

1. Source Tree

⇒ here: **mainline kernel** (torvalds/Linux)



2. Revision

⇒ here: **all releases** in 2002–2024 (i.e., KConfig)



3. Architecture

⇒ here: **all architectures** (except for um)

Architecture	Subsumed Architecture
alpha	
arc	
arm, arm64	arm26
csky	
hexagon	
loongarch	
m68k (m68000)	m68knommu
microblaze	
mips	mips64
nios2	
openrisc	
parisc	parisc64
powerpc	ppc, ppc64
riscv	
s390 (z Systems)	s390x
sh (SuperH)	sh64
sparc	sparc32, sparc64
um (User Mode Linux)	
x86	i386, x86_64
xtensa	

Choosing and **Analyzing** the Feature Model

“are we analyzing the model **the same way?**”

Choosing and Analyzing the Feature Model

```
config X86_32
    def_bool y
    depends on !64BIT
    select ARCH_WANT_IPC_PARSE_VERSION

config SMP
    bool "Symmetric multi-processing support"
    help
        This enables support for systems with
        more than one CPU.

if X86_32
    config X86_BIGSMP
        bool "Support for big SMP systems"
        depends on SMP
        help
            This option is needed for the systems
            that have more than 8 CPUs.
endif # X86_32
```

“are we analyzing the model **the same way**?”

4. Extraction

⇒ here: **KConfigReader** and **KClause**

$$\begin{aligned} F &= \{X86_32, 64BIT, SMP, X86_BIGSMP\} \\ \phi &= (X86_32 \rightarrow \neg 64BIT) \\ &\quad \wedge (X86_BIGSMP \rightarrow (X86_32 \wedge SMP)) \end{aligned}$$

Choosing and **Analyzing** the Feature Model

CNF Transformation

[Kuijter et al. 2022]

$$\phi_{CNF} := tseitinCNF(\phi)$$

Backbone Transformation

[Biere et al. 2023]

$$V_{dead} := \{v \in V \mid \neg SAT(\phi_{CNF} \wedge v)\}$$

$$V_{core} := \{v \in V \mid \neg SAT(\phi_{CNF} \wedge \neg v)\}$$

$$\phi_{CNF}^{back} := \phi_{CNF} \wedge \bigwedge_{v \in V_{dead}} \neg v \wedge \bigwedge_{v \in V_{core}} v$$

“are we analyzing the model **the same way**?”

4. Extraction

⇒ here: **KConfigReader** and **KClause**



5. Transformation

⇒ here: **CNF** and **backbone transformation**

Choosing and Analyzing the Feature Model



#Features

$|V|$ (sort of)



#Configurations

$\#SAT(V, \phi)$ (sort of)

(more on this later)

“are we analyzing the model **the same way**?”

4. Extraction

⇒ here: **KConfigReader** and **KClause**



5. Transformation

⇒ here: **CNF** and **backbone transformation**



6. Analysis

⇒ here: **#features** and **#configurations**

Evaluation

Choosing and Analyzing the Feature Model

1. Source Tree

only mainline kernel

2. Revision

releases in 2002–2024

3. Architecture

all but um

4. Extraction

KConfigReader,
KClause

5. Transformation

CNF, backbone

6. Analysis

#features,
#configurations

Research Questions

Evaluation

Choosing and Analyzing the Feature Model

1. Source Tree

only mainline kernel

2. Revision

releases in 2002–2024

3. Architecture

all but um

4. Extraction

KConfigReader,
KClause

5. Transformation

CNF, backbone

6. Analysis

#features,
#configurations

Research Questions

RQ₁ When **can we count** features + configurations?

Evaluation

Choosing and Analyzing the Feature Model

1. Source Tree

only mainline kernel

2. Revision

releases in 2002–2024

3. Architecture

all but um

4. Extraction

KConfigReader,
KClause

5. Transformation

CNF, backbone

6. Analysis

#features,
#configurations

Research Questions

RQ₁ When **can we count** features + configurations?

RQ₂₋₃ How **configurable** is the Linux kernel?

Evaluation

Choosing and Analyzing the Feature Model

1. Source Tree

only mainline kernel

2. Revision

releases in 2002–2024

3. Architecture

all but um

4. Extraction

KConfigReader,
KClause

5. Transformation

CNF, backbone

6. Analysis

#features,
#configurations

Research Questions

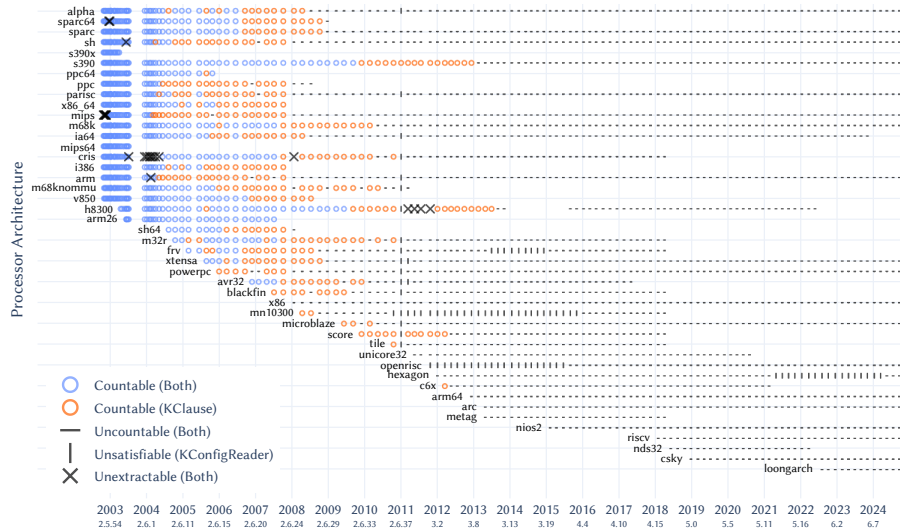
RQ₁ When **can we count** features + configurations?

RQ₂₋₃ How **configurable** is the Linux kernel?

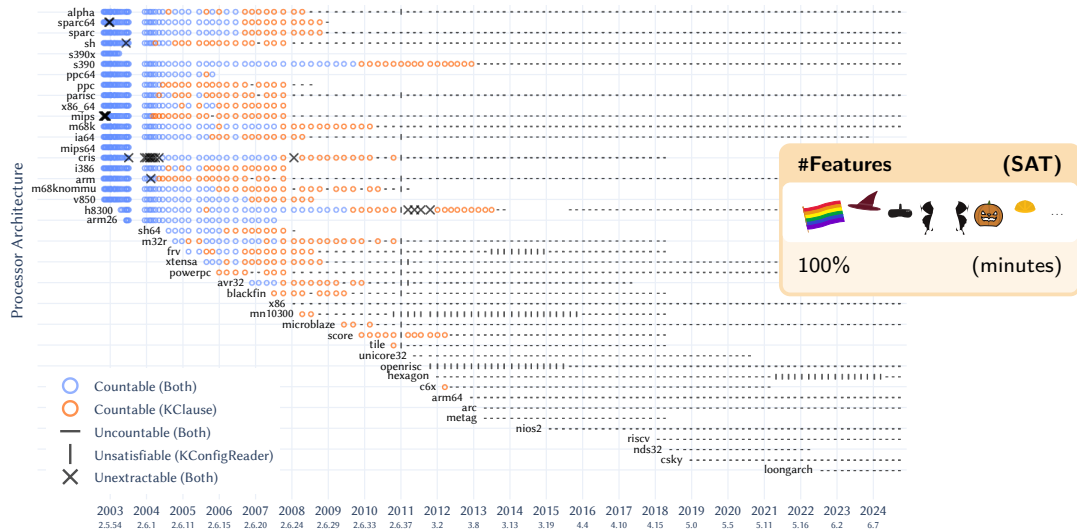
RQ₃₋₄ How does its configurability **evolve**?

RQ₁: When Can We Count Features and Configurations?

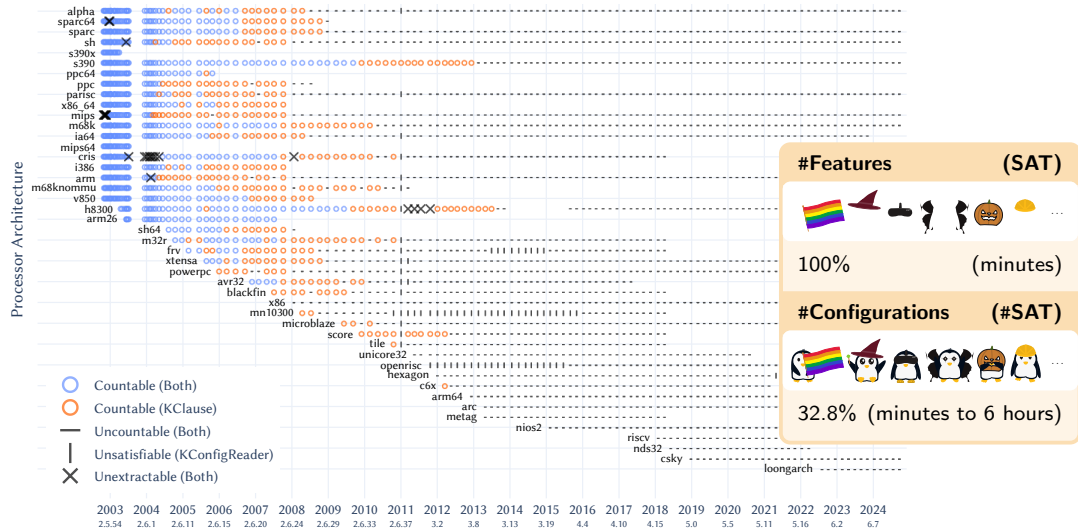
RQ₁: When Can We Count Features and Configurations?



RQ₁: When Can We Count Features and Configurations?

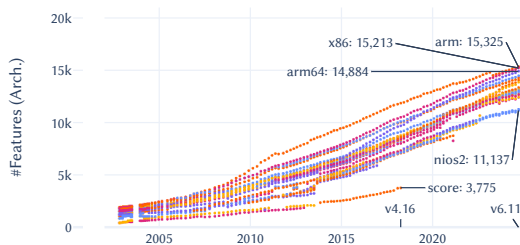


RQ₁: When Can We Count Features and Configurations?



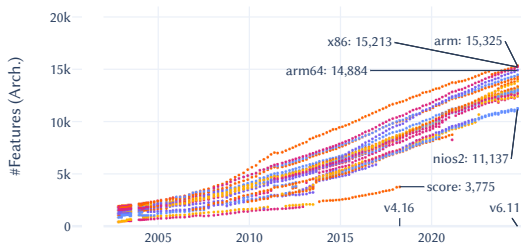
RQ₂₋₃: How Configurable is the Linux Kernel?

RQ₂₋₃: How Configurable is the Linux Kernel?



RQ₂₋₃: How Configurable is the Linux Kernel?

- auxiliary variables
- non-related vars
- dead features
- + unconstrained features
- non-Boolean variables



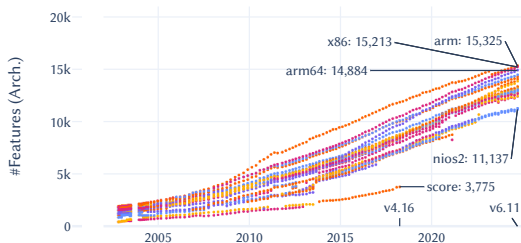
RQ₂₋₃: How Configurable is the Linux Kernel?

- auxiliary variables
- non-related vars
- dead features
- + unconstrained features
- non-Boolean variables

sustained **linear growth** (still!)

a given **architecture** only contains half of the features

does not depend on **extractor**



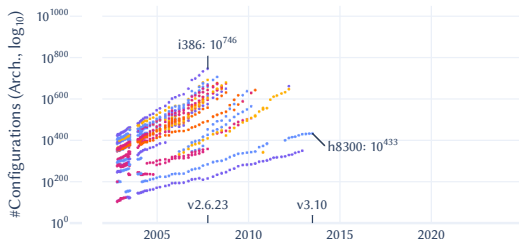
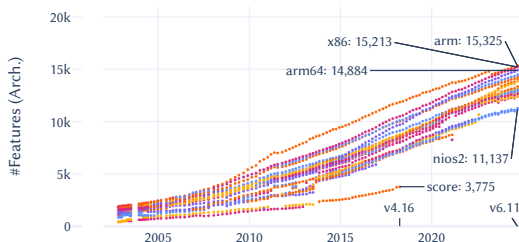
RQ₂₋₃: How Configurable is the Linux Kernel?

- auxiliary variables
- non-related vars
- dead features
- + unconstrained features
- non-Boolean variables

sustained **linear**
growth (still!)

a given **architecture**
only contains half of
the features

does not depend on **ex-
tractor**



RQ₂₋₃: How Configurable is the Linux Kernel?

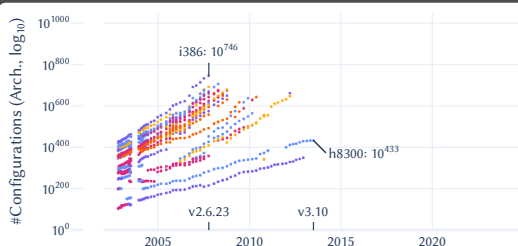
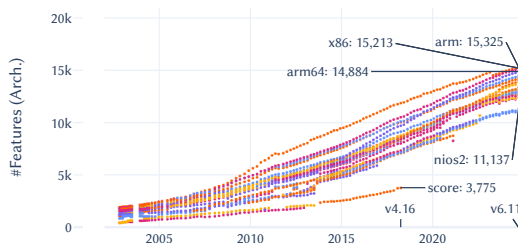
- auxiliary variables
- non-related vars
- dead features
- + unconstrained features
- non-Boolean variables

+ unconstrained features

sustained **linear growth** (still!)

a given **architecture** only contains half of the features

does not depend on **extractor**



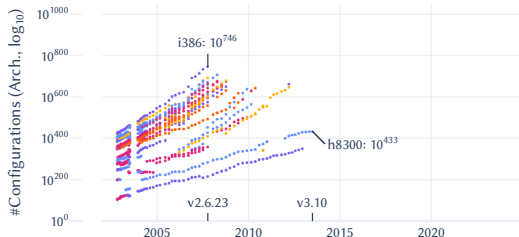
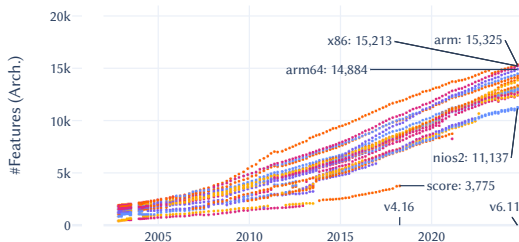
RQ₂₋₃: How Configurable is the Linux Kernel?

- auxiliary variables
- non-related vars
- dead features
- + unconstrained features
- non-Boolean variables

sustained **linear growth** (still!)

a given **architecture** only contains half of the features

does not depend on **extractor**



+ unconstrained features

35.8% successes
(mostly ≤ 2007)

exponential growth

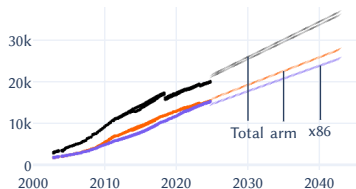
only $1/10^{130}$ configurations are **valid**

depends on **extractor**
($\log_{10} * 1.5$)

pushes the limits of
state-of-the-art #SAT
solvers

RQ₄₋₅: How Does the Linux Kernel's Configurability Evolve?

RQ₄₋₅: How Does the Linux Kernel's Configurability Evolve?



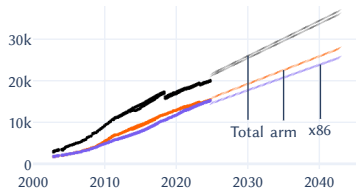
(a) #Features (by Time)

Total #Features

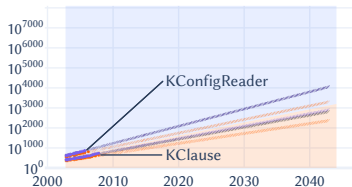


per week	+ 16
per release	+ 220 – 70
today	20,000
in 20 years	37,000 – 38,000

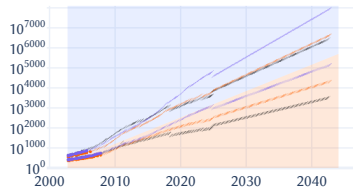
RQ₄₋₅: How Does the Linux Kernel's Configurability Evolve?



(a) #Features (by Time)



(b) #Configurations ($\log_{10}$, by Time)



(c) #Configurations ($\log_{10}$, by Features)

Total #Features



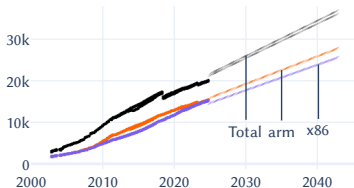
per week + 16
 per release + 220 – 70
 today 20,000
 in 20 years 37,000 – 38,000

Total #Configurations

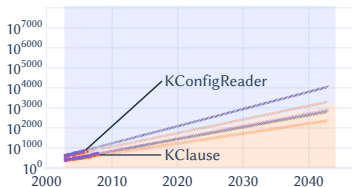


per week * 10
 per release * 100
 today $10^{1,600}$ – $10^{3,600}$
 in 20 years $10^{2,800}$ – $10^{6,400}$

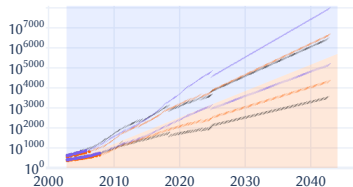
RQ₄₋₅: How Does the Linux Kernel's Configurability Evolve?



(a) #Features (by Time)



(b) #Configurations ($\log_{10}$, by Time)



(c) #Configurations ($\log_{10}$, by Features)

Total #Features



per week + 16
 per release + 220 – 70
 today 20,000
 in 20 years 37,000 – 38,000

Total #Configurations



per week * 10
 per release * 100
 today $10^{1,600} - 10^{3,600}$
 in 20 years $10^{2,800} - 10^{6,400}$

Key Insights

- disagreeing definitions
 - exponential growth
 - Linux = largest product line
 - architectures vary
- ⇒ improve SAT/#SAT?
 ⇒ reduce variability?

What's Next?

What's Next?

BusyBox axTLS Buildroot Em-
bToolkit Freetz-NG L4Re toy-
box uClibc **Linux** uClibc-ng core-
boot u-boot Crosstool-NG Soletta
FreeWRT QEMU RIOT Unikraft
Zephyr esp-idf sdk-nrf ...

How configurable is
other system software?



What's Next?

BusyBox axTLS Buildroot Em-
bToolkit Freetz-NG L4Re toy-
box uClibc **Linux** uClibc-ng core-
boot u-boot Crosstool-NG Soletta
FreeWRT QEMU RIOT Unikraft
Zephyr esp-idf sdk-nrf ...

How configurable is
other system software?



We don't know (yet)!
But we're on it.



wip

What's Next?

BusyBox axTLS Buildroot Em-
bToolkit Freetz-NG L4Re toy-
box uClibc **Linux** uClibc-ng core-
boot u-boot Crosstool-NG Soletta
FreeWRT QEMU RIOT Unikraft
Zephyr esp-idf sdk-nrf ...

How configurable is
other system software?



Can **SAT solvers** keep up with
the Linux kernel's feature model?

We don't know (yet)!
But we're on it.



wip

What's Next?

BusyBox axTLS Buildroot Em-
bToolkit Freetz-NG L4Re toy-
box uClibc **Linux** uClibc-ng core-
boot u-boot Crosstool-NG Soletta
FreeWRT QEMU RIOT Unikraft
Zephyr esp-idf sdk-nrf ...

How configurable is
other system software?

Can **SAT solvers** keep up with
the Linux kernel's feature model?

We don't know (yet)!
But we're on it.

No, not really.

wip

ICSE '26

What's Next?

BusyBox axTLS Buildroot Em-
bToolkit Freetz-NG L4Re toy-
box uClibc **Linux** uClibc-ng core-
boot u-boot Crosstool-NG Soletta
FreeWRT QEMU RIOT Unikraft
Zephyr esp-idf sdk-nrf ...

How configurable is
other system software?

Can **SAT solvers** keep up with
the Linux kernel's feature model?

We don't know (yet)!
But we're on it.

No, not really.

wip

ICSE '26

What's Next?

BusyBox axTLS Buildroot Em-
bToolkit Freetz-NG L4Re toy-
box uClibc **Linux** uClibc-ng core-
boot u-boot Crosstool-NG Soletta
FreeWRT QEMU RIOT Unikraft
Zephyr esp-idf sdk-nrf ...

How configurable is
other system software?

Can **SAT solvers** keep up with
the Linux kernel's feature model?

We don't know (yet)!
But we're on it.

No, not really.

wip

ICSE '26

Is there no simple solution?

Conclusion

Date Sun, 1 Apr 2012 00:33:21 +0800

[LKML 2012]

From Paul E. McKenney

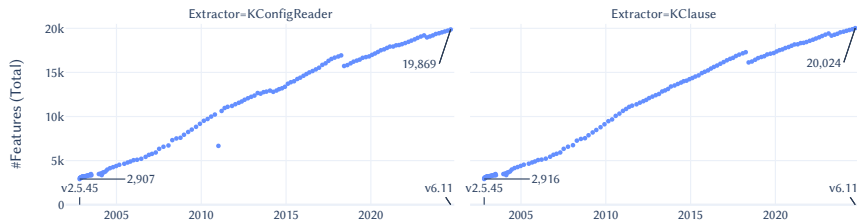
Subject [PATCH RFC] Simplify the Linux kernel by reducing its state space

Although there have been numerous complaints about the complexity of parallel programming, the plain truth is that the incremental complexity of parallel programming over that of sequential programming is not as large as is commonly believed. Despite that you might have heard, the mind-numbing complexity of modern computer systems is not due so much to there being multiple CPUs, but rather to there being any CPUs at all. In short, for the ultimate in computer-system simplicity, the optimal choice is NR_CPUS=0. This commit therefore limits kernel builds to zero CPUs. This change has the beneficial side effect of rendering all kernel bugs harmless. Furthermore, this commit enables additional beneficial changes, for example, the removal of those parts of the kernel that are not needed when there are zero CPUs.

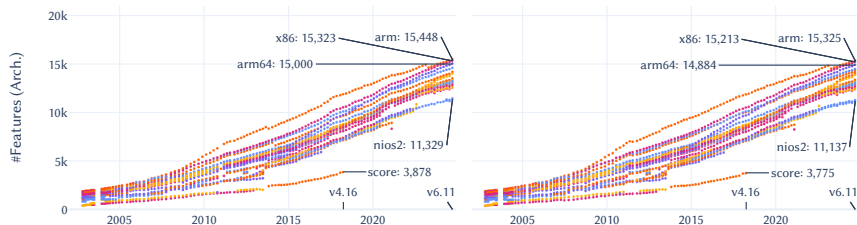


Thank you for listening!

Number of Features

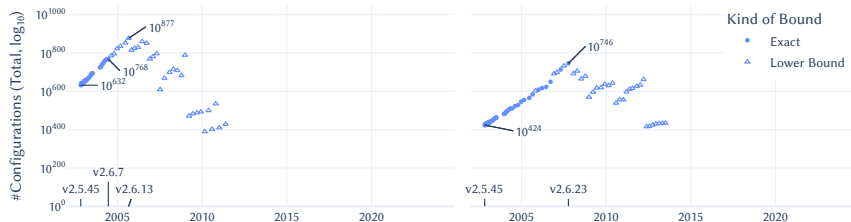


(a) Total Number of Features

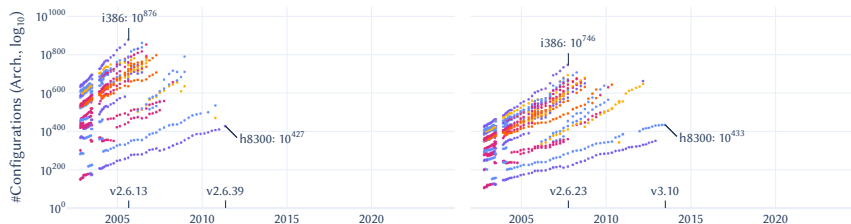


(b) Number of Features per Architecture

Number of Configurations

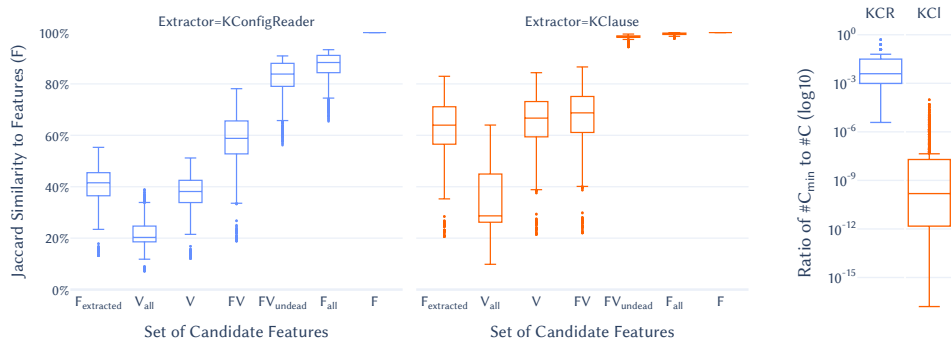


(c) Total Number of Configurations



(d) Number of Configurations per Architecture

Feature and Configuration Candidates



Added and Removed Features and Configurations

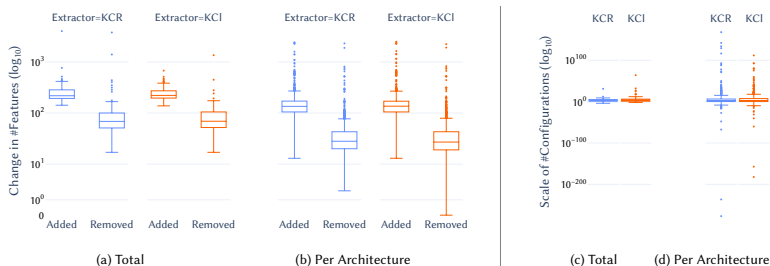
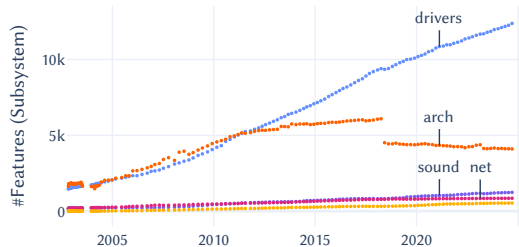


Fig. 5. Change in the number of features (5a–b, change in absolutes) and configurations (5c–d, change in scale) per kernel release.

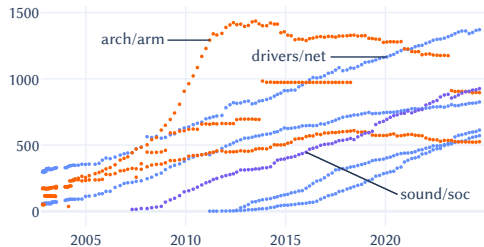
Table 3. Quartiles of changes shown in Figure 5, rounded to integers.

Quartile	Number of Features								Number of Configurations			
	(a) Total				(b) Per Architecture				(c) Total		(d) Per Architecture	
	KCONFIGREADER		KCLAUSE		KCONFIGREADER		KCLAUSE		KCR	KCL	KCR	KCL
Q4 (Maximum)	+ 4,005	− 3,772	+ 676	− 1,355	+ 2,406	− 2,303	+ 2,492	− 2,231	· 10 ³¹	· 10 ⁶⁴	· 10 ¹⁶⁸	· 10 ¹¹²
Q3	+ 284	− 99	+ 272	− 104	+ 171	− 43	+ 170	− 43	· 10 ⁵	· 10 ⁶	· 10 ⁶	· 10 ⁷
Q2 (Median)	+ 218	− 68	+ 221	− 69	+ 135	− 28	+ 136	− 27	· 10 ²	· 10 ²	· 10 ²	· 10 ²
Q1	+ 194	− 52	+ 196	− 52	+ 105	− 20	+ 105	− 19	· 10 ¹	· 10 ¹	· 10 ⁰	· 10 ⁰
Q0 (Minimum)	+ 142	− 17	+ 138	− 17	+ 13	− 3	+ 13	− 1	· 10 ⁻⁴	· 10 ⁻²	· 10 ⁻²⁷⁷	· 10 ⁻¹⁸²

Number of Features per Subsystem

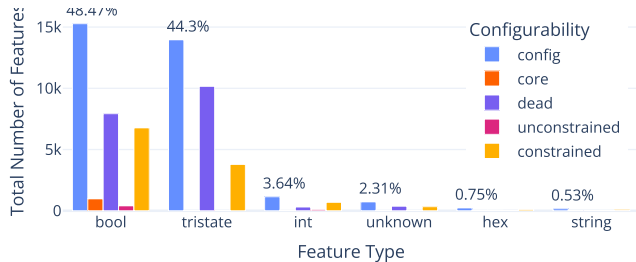


(a) Top-Level Subsystems

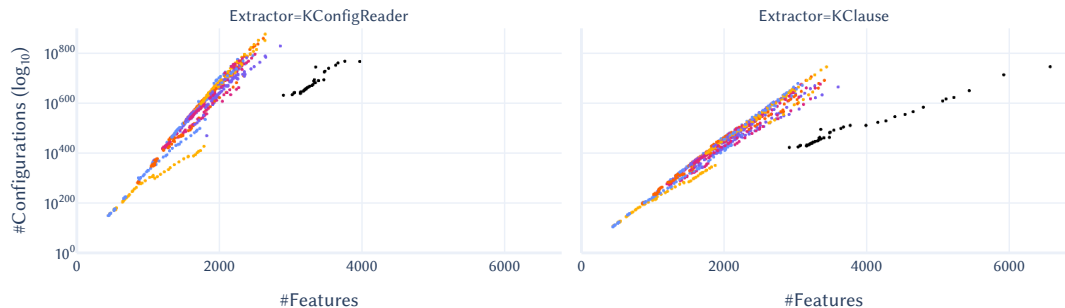


(b) Second-Level Subsystems

Feature Types



Correlation Between Features and Configurations



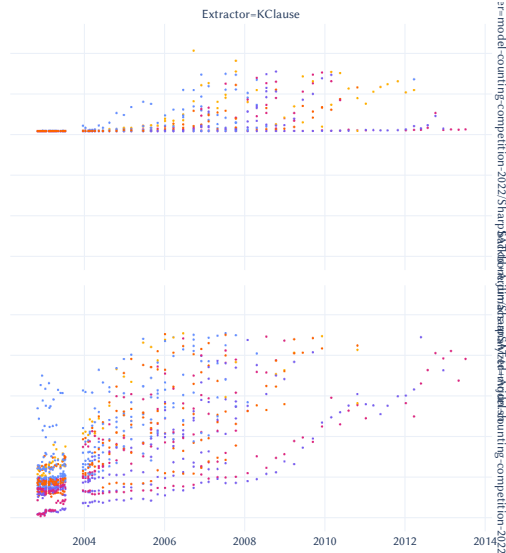
[illegible]

Image Credits

- **Tux, the Linux penguin**, Larry Ewing and The GIMP [https://commons.wikimedia.org/wiki/File:Tux.png]
- **Microsoft Excel**, FreddyMendozaRueda [https://commons.wikimedia.org/wiki/File:Crear-tablas-en-excel-0000161.png]
- **XO Sugar**, Firefoxman [https://commons.wikimedia.org/wiki/File:XO-sugar.png]
- **Sonic Adventure**, Sonic Team [https://en.wikipedia.org/wiki/File:Sonic_Adventure_Dreamcast.png]
- **NAT Braille**, Raminala [https://commons.wikimedia.org/wiki/File:CaptureNAT.png]
- **Android 16**, Paowee [https://commons.wikimedia.org/wiki/File:Android_16_home_screen_screenshot.png]
- **Toshiba Satellite S40t**, Cheon Fong Liew [https://commons.wikimedia.org/wiki/File:Toshiba_Satellite_S40t_laptop_-_10087025595.jpg]
- **XO Laptop**, One Laptop per Child [https://commons.wikimedia.org/wiki/File:Xo_intro_v2.jpg]
- **EuroBraille Iris**, Mfaure [https://commons.wikimedia.org/wiki/File:Plage-braille-avec-touches-speciales.jpg]
- **Bugdroid**, Irina Blok [https://commons.wikimedia.org/wiki/File:Android_2023_3D_logo_and_wordmark.svg]
- **Sega Dreamcast**, Evan-Amos [https://commons.wikimedia.org/wiki/File:Dreamcast-Console-Set.jpg]
- **TikZpingus**, Florian Sihler [https://github.com/EagleoutIce/tikzpingus]
- **Sandwiches**, Paul Maximilian Bittner [https://doi.org/10.18725/OPARU-58798]

All images were used with permission. The slides were created with FancyBeamer and draw.io.